

Aprendizaje estadístico para confiabilidad

Predicción de fallas con datos censurados

Pedro Luiz Ramos

Pontificia Universidad Católica de Chile

Del dato a la decisión de mantenimiento

20 min	**Datos, problema predictivo y supervivencia** Backblaze, censura, Kaplan–Meier y C-index.	1–17
18 min	**Modelos de regresión** AFT, Cox y regularización LASSO/Ridge.	18–26
25 min	**Aprendizaje flexible** Árboles, RSF, boosting y SVM para datos censurados.	27–38
22 min	**Aplicación: fallas de discos duros** Ajustes en R, comparación y lectura de resultados.	39–49
5 min	**Síntesis y discusión**	50–51

El problema de mantenimiento

En un centro de almacenamiento, los discos envejecen mientras el servicio debe continuar. Reemplazarlos demasiado pronto puede desperdiciar recursos; esperar hasta la falla deja poco margen para intervenir.

La información de cada disco permite anticipar fallas, ordenar los equipos según su riesgo y apoyar la planificación del mantenimiento.

El seguimiento es incompleto: algunos discos fallan y otros siguen operativos al terminar la observación. Este escenario motiva el análisis de supervivencia con un enfoque predictivo.



Imagen ilustrativa; no representa las instalaciones de Backblaze.

Los datos: discos duros de Backblaze

Registros diarios de discos en operación en centros de almacenamiento.

Cantidad	Valor
Registros diarios	5 091 501
Discos distintos	29 072
Fallas observadas	724
Sin falla registrada	28 348
Porcentaje sin falla	97.51 %
Seguimiento máximo	266 días

Ventana de observación

10 de abril–31 de diciembre de 2013.

Falla registrada

Primera falla registrada del disco.

Predictores

Fabricante, capacidad y atributos SMART de operación y deterioro.

Predominan los discos sin una falla registrada.

Datos públicos: Backblaze Drive Stats. Resúmen del análisis existente.

De registros diarios a una fila por disco

Cada disco se identifica por su número de serie. Sus registros diarios se resumen mediante tres fechas: ingreso, primera falla y última observación.

$$d_i^* = \begin{cases} d_{i,\text{falla}}, & \text{si se registra falla,} \\ d_{i,\text{final}}, & \text{en otro caso,} \end{cases} \qquad Y_i = d_i^* - d_{i,\text{ingreso}} + 1.$$

- Una fila resume el seguimiento de un disco; Y_i cuenta sus días observados.
- Las horas SMART describen uso previo, pero no redefinen el origen temporal.
- Se conserva además si el seguimiento terminó con una falla registrada.

El objetivo es tiempo hasta falla desde el ingreso, no vida total del equipo.

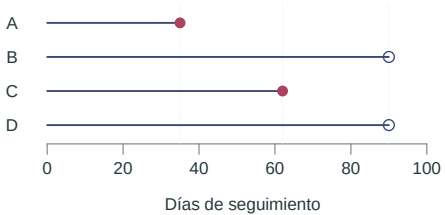
La información disponible de cada disco

Información	Representación	Interpretación
Hardware	Fabricante; capacidad	Familia de hardware
Horas de uso	$\log(1 + \text{horas})$	Exposición previa
Temperatura	Grados Celsius	Estado operativo
Reasignados	$\log(1 + n)$ y $\mathbb{I}(n > 0)$	Deterioro del medio
Pendientes	$\mathbb{I}(n > 0)$	Inestabilidad
Fecha de ingreso	Días desde inicio de ventana	Contexto temporal

Los atributos SMART son registros internos de operación y estado. Para predecir al ingreso, se requiere información disponible en ese momento.

De los datos a la supervivencia

El resultado combina **tiempo de seguimiento** y **estado al finalizar**: falla registrada o disco sin falla observada.



Punto lleno: falla; círculo vacío: censura. Ejemplo didáctico.

Formalización

$$Y_i = \min(T_i, C_i),$$
$$\Delta_i = \mathbb{I}(T_i \leq C_i).$$

T_i: tiempo hasta falla.
C_i: tiempo hasta censura.
Si el disco no falla antes de salir, solo se sabe que $T_i > C_i$.

La censura a la derecha motiva el análisis de supervivencia.

El enfoque predictivo con covariables

Las covariables permiten anticipar fallas desde el ingreso.

Objetivo	Cantidad de interés	Uso
Ordenar por riesgo	$r(x)$: puntaje de riesgo	Priorizar revisión
Falla antes de τ	$1 - S(\tau \mid x)$	Planificar
Tiempo hasta falla	$t_{0.5}(x) = \inf\{t : S(t \mid x) \leq .5\}$	Resumir tiempo

$S(t \mid x) = P(T > t \mid X = x)$: confiabilidad condicional.

El criterio principal será la concordancia (C-index).

El ordenamiento no asegura probabilidades calibradas ni tiempos individuales precisos.

Un supuesto para analizar la censura

La formulación usual supone censura independiente, condicional en las covariables disponibles:

$$T \perp C \mid X, \qquad P(C \geq t \mid X = x) > 0 \quad \text{en el horizonte de interés.}$$

- Cierre administrativo: todos los discos se observan hasta una fecha fijada.
- Retiro por deterioro: puede generar censura informativa si no se registra su motivo.
- En Backblaze, el origen es la **primera observación**, no la instalación.

La predicción se refiere a discos que ya pertenecen a esta cohorte.

La independencia de la censura es una hipótesis; no se demuestra observando solo (Y, Δ, X).

Confiabilidad, riesgo y riesgo acumulado

Para un tiempo continuo no negativo, se definen

$$S(t \mid x) = P(T > t \mid X = x), \qquad f(t \mid x) = -\frac{dS(t \mid x)}{dt},$$

$$h(t \mid x) = \lim_{u \downarrow 0} \frac{P(t \le T < t + u \mid T \ge t, x)}{u} = \frac{f(t \mid x)}{S(t \mid x)}.$$

Entonces

$$H(t \mid x) = \int_0^t h(u \mid x)\, du, \qquad S(t \mid x) = e^{-H(t|x)}.$$

$h(t \mid x)$ **es una tasa instantánea, no una probabilidad de falla.**

La verosimilitud incorpora la censura

Omitiendo el mecanismo independiente de censura:

$$L(\theta) = \prod_{i=1}^{n} f_\theta(Y_i \mid x_i)^{\Delta_i} S_\theta(Y_i \mid x_i)^{1-\Delta_i}.$$

Equivalentemente,

$$\ell(\theta) = \sum_{i=1}^{n} \{\Delta_i \log h_\theta(Y_i \mid x_i) - H_\theta(Y_i \mid x_i)\}.$$

- Falla: contribuye la densidad en el tiempo observado.
- Censura: contribuye la probabilidad de superar ese tiempo.

Eliminar los censurados descarta información.

Kaplan–Meier, paso a paso

En los tiempos distintos de falla $t_1 < \cdots < t_J$, sean d_j las fallas y n_j los discos en riesgo justo antes de t_j:

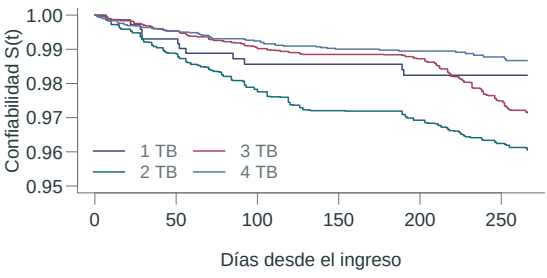
$$\widehat{S}_{KM}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right).$$

Ejemplo con seis discos
En $t = 2$ falla un disco: $n_1 = 6$.
En $t = 3$ se censura otro disco.
En $t = 5$ falla uno de los cuatro restantes.

$$\widehat{S}_{KM}(5) = \left(1 - \frac{1}{6}\right)\left(1 - \frac{1}{4}\right) = 0.625.$$

La censura reduce el conjunto en riesgo, pero no produce un salto de la curva.

Una primera mirada: curvas por capacidad



Las diferencias son **marginales**: capacidad, fabricante y antigüedad pueden estar asociados.

La escala vertical se restringe a $[0.95, 1]$ para mostrar diferencias pequeñas.

```
km <- survival::survfit(Surv(time_days, event) ~ capacity_group, data = datos)
```

Cohorte completa Backblaze 2013; sin bandas ni marcas de censura para facilitar la lectura.

Tres conjuntos, tres funciones diferentes

Conjunto	Discos	Fallas	Censuras
Entrenamiento (60 %)	17 442	434	17 008
Validación (20 %)	5 815	145	5 670
Test (20 %)	5 815	145	5 670

- **Entrenamiento**: estimar parámetros y construir modelos.
- **Validación**: escoger hiperparámetros y configuración.
- **Test**: estimar desempeño de los ajustes ya seleccionados.

Partición aleatoria estratificada por evento; semilla 20260704. No es validación temporal ni externa.

C-index: comparar pares observables

Sea r_i un puntaje donde un valor mayor indica **mayor riesgo**. Se define

$$A_{ij} = \Delta_i \mathbb{I}(Y_i < Y_j).$$

El estimador de Harrell, con medio crédito para empates del puntaje, es

$$\widehat{C}_H = \frac{\sum_{i \neq j} A_{ij} \left[\mathbb{I}(r_i > r_j) + \frac{1}{2}\mathbb{I}(r_i = r_j) \right]}{\sum_{i \neq j} A_{ij}}.$$

- El primer tiempo debe corresponder a una falla observada.
- $C \approx 0.5$: sin ordenamiento útil; $C = 1$: concordancia perfecta.

Con censura alta, importa qué pares permanecen observables.

Expresión para tiempos de evento no empatados; el software trata los empates temporales. Harrell et al. (1996).

Un C-index calculado a mano

Disco	Y_i	Δ_i	r_i
A	35	1	.9
B	90	0	.2
C	62	1	.3
D	90	0	.4

Cinco pares comparables
A–B, A–C, A–D, C–B, C–D.
Los cuatro primeros son concordantes.
C–D es discordante: C falla antes, pero $r_C < r_D$.

$$\widehat{C}_H = \frac{4}{5} = 0.80.$$

B–D no es comparable: ninguno tiene una falla observada.

Orientar todos los scores hacia riesgo

```
survival::concordance(
  Surv(time_days, event) ~ riesgo,
  data = test, reverse = TRUE
)$concordance
```

Salida del modelo	Puntaje usado para C-index
Cox / boosting	Predictor lineal o log-riesgo
RSF	Mortalidad acumulada predicha
AFT / SVM de regresión	Menos tiempo predicho: $r_i = -\widehat{T}_i$

$C = 0.844$ **no significa “84.4 % de fallas detectadas”.**

AFT: modelar la escala del tiempo

El modelo de tiempo de falla acelerado plantea

$$\log T = \eta(x) + \sigma\varepsilon, \qquad \eta(x) = \beta_0 + x^\top \beta, \quad \sigma > 0.$$

Para dos perfiles que difieren solo en una unidad de x_j,

$$\frac{t_p(x + e_j)}{t_p(x)} = e^{\beta_j},$$

donde $t_p(x)$ es el cuantil p del tiempo hasta falla.

$e^{\beta_j} > 1$ **estira el tiempo;** $e^{\beta_j} < 1$ **lo acorta.**

Ejemplo: si $e^{\beta_j} = 1.5$, una mediana de 80 días pasa a 120 días, manteniendo las demás covariables.

AFT Weibull y log-normal

Weibull: error de valor extremo mínimo

$$S(t \mid x) = \exp\left[-\left\{\frac{t}{e^{\eta(x)}}\right\}^{1/\sigma}\right], \qquad k = 1/\sigma.$$

Riesgo decreciente si $k < 1$, constante si $k = 1$, creciente si $k > 1$.

Log-normal: $\varepsilon \sim N(0,1)$

$$S(t \mid x) = 1 - \Phi\left(\frac{\log t - \eta(x)}{\sigma}\right), \qquad t_{0.5}(x) = e^{\eta(x)}.$$

Permite un riesgo que primero aumenta y luego disminuye.

La distribución elegida condiciona la extrapolación más allá del seguimiento.

Estimar AFT con datos censurados

Con $z_i = (\log Y_i - \eta_i)/\sigma$,

$$\ell(\beta_0, \beta, \sigma) = \sum_i \left[\Delta_i \left\{\log f_\varepsilon(z_i) - \log(\sigma Y_i)\right\} + (1 - \Delta_i) \log S_\varepsilon(z_i)\right].$$

La optimización numérica estima conjuntamente regresión y escala.

```
aft <- survival::survreg(formula_surv, data = train, dist = "lognormal")
tiempo <- predict(aft, newdata = test, type = "quantile", p = 0.5)
riesgo <- -tiempo
```

En la aplicación: Weibull y log-normal sin penalización.

`formula_surv` representa `Surv(time_days, event) ~ covariables`.

AFT penalizado: controlar complejidad

Una extensión natural estima

$$(\widehat{\beta}_0, \widehat{\beta}, \widehat{\sigma}) = \arg \min_{\beta_0, \beta, \sigma > 0} \left\{ -\frac{1}{n} \ell(\beta_0, \beta, \sigma) + \lambda P_\alpha(\beta) \right\},$$

$$P_\alpha(\beta) = \alpha \sum_{j=1}^{p} |\beta_j| + \frac{1-\alpha}{2} \sum_{j=1}^{p} \beta_j^2.$$

- LASSO ($\alpha = 1$): puede llevar coeficientes exactamente a cero.
- Ridge ($\alpha = 0$): contrae coeficientes y estabiliza predictores correlacionados.
- No se penalizan intercepto ni escala; λ se selecciona fuera del ajuste.

Extensión metodológica del curso; no se reportan resultados AFT penalizados en este estudio.

Cox: riesgo basal y covariables

El modelo de riesgos proporcionales especifica

$$h(t \mid x) = h_0(t) e^{x^\top \beta}.$$

Para dos perfiles x_a y x_b,

$$\mathrm{HR}(x_a, x_b) = \frac{h(t \mid x_a)}{h(t \mid x_b)} = \exp\{(x_a - x_b)^\top \beta\}.$$

- $h_0(t)$ queda sin especificar: modelo semiparamétrico.
- La razón de riesgos no depende del tiempo bajo este supuesto.
- e^{β_j} es un multiplicador de la tasa, no del tiempo ni de la probabilidad.

AFT interpreta cambios de tiempo; Cox interpreta cambios de riesgo.

Cox: la verosimilitud parcial

Sin empates de falla, con $\mathcal{R}_i = \{j : Y_j \geq Y_i\}$,

$$L_P(\beta) = \prod_{i:\Delta_i=1} \frac{e^{x_i^\top \beta}}{\sum_{j\in\mathcal{R}_i} e^{x_j^\top \beta}},$$

$$\ell_P(\beta) = \sum_{i:\Delta_i=1} \left[x_i^\top \beta - \log \sum_{j\in\mathcal{R}_i} e^{x_j^\top \beta} \right].$$

Cada falla compara su disco con los que aún estaban observados y operativos.

Los censurados participan en los conjuntos de riesgo hasta su salida.

Cox (1972). Tiempos diarios generan empates: `coxph` usa Efron y el ajuste `glmnet` de este estudio usa Breslow.

Del score de Cox a una curva

Una vez estimado β, el estimador de Breslow del riesgo basal acumulado es

$$\widehat{H}_0(t) = \sum_{t_j \le t} \frac{d_j}{\sum_{i \in \mathcal{R}_j} e^{x_i^\top \widehat{\beta}}}, \qquad \widehat{S}(t \mid x) = \exp\{-\widehat{H}_0(t) e^{x^\top \widehat{\beta}}\}.$$

```
cox <- survival::coxph(formula_surv, data = train, ties = "efron", x = TRUE)
riesgo <- predict(cox, newdata = test, type = "lp")
curva <- survival::survfit(cox, newdata = perfil)
```

La expresión muestra Breslow; el software adapta el tratamiento de empates a la configuración del ajuste.

Para ordenar basta el score; para probabilidades hace falta el riesgo basal.

Cox penalizado: LASSO y Ridge

La estimación combina la evidencia parcial con una penalización:

$$\widehat{\beta}_{\lambda,\alpha} = \arg\min_{\beta} \left[-\frac{1}{n}\ell_P(\beta) + \lambda \left\{ \alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2 \right\} \right].$$

LASSO: $\alpha = 1$
Selección y contracción.
Puede elegir una entre covariables muy correlacionadas.

Ridge: $\alpha = 0$
Contracción sin selección exacta.
Distribuye el efecto entre predictores correlacionados.

Mayor λ implica más regularización, no necesariamente mayor C-index.

Tibshirani (1997); Simon et al. (2011). La normalización de la pérdida depende de la implementación.

Elegir λ sin mirar el test

$$\widehat{\lambda} = \arg\max_{\lambda \in \Lambda} \widehat{C}_H(\text{validación}; \widehat{\beta}_{\lambda,\alpha}).$$

```
ajuste <- glmnet::glmnet(x_train, Surv(train$time_days, train$event),
                         family = "cox", alpha = 1, nlambda = 80,
                         standardize = TRUE, cox.ties = "breslow")
lp_valid <- predict(ajuste, newx = x_valid, type = "link")
# Comparar el C-index de las columnas y fijar lambda_elegido.
lp_test <- predict(ajuste, newx = x_test, s = lambda_elegido, type = "link")
```

En este análisis: $\lambda_{LASSO} = 0.000445$ y $\lambda_{Ridge} = 0.00471$.

El test se usa después de cerrar la selección.

Árboles: separar perfiles de riesgo

Un corte divide el nodo en L y R; el criterio log-rank utiliza

$$U = \sum_j \left(d_{Lj} - \frac{n_{Lj}}{n_j} d_j \right), \qquad V = \sum_j \frac{n_{Lj} n_{Rj} d_j (n_j - d_j)}{n_j^2 (n_j - 1)}.$$

$$s^* = \arg\max_{s \in \mathcal{S}} \frac{U(s)^2}{V(s)}.$$

- Se buscan cortes en distintas covariables y se repite recursivamente.
- La regla final es interpretable, pero pequeños cambios pueden alterar el árbol.

El árbol ajustado usa verosimilitud, no log-rank.

$n_j = n_{Lj} + n_{Rj}$; se suman tiempos con varianza definida. El criterio de `rpart(method="exp")` se presenta a continuación.

El árbol ajustado: criterio y poda

`rpart(method="exp")` reescala los tiempos con un riesgo basal estimado y utiliza una formulación tipo Poisson para el riesgo relativo de cada nodo.

$$\ell_g(\lambda_g) = \sum_{i \in g} \{\Delta_i \log \lambda_g - \lambda_g z_i\} + \text{cte}, \quad z_i = \widetilde{H}_0(Y_i), \quad \widehat{\lambda}_g = \frac{\sum_{i \in g} \Delta_i}{\sum_{i \in g} z_i}.$$

La expresión ilustra la estimación sin contracción; la implementación incorpora regularización del riesgo del nodo.

```
arbol <- rpart::rpart(formula_surv, data = train, method = "exp",
  control = rpart::rpart.control(cp = 0.001, minbucket = 80, maxdepth = 5))
arbol <- rpart::prune(arbol, cp = cp_elegido)
```

Se podó por el menor error de validación cruzada de `rpart`.

Criterio: ganancia de desviación; $cp = 0.00224$. Documentación de `rpart.exp`.

RSF: agregar árboles de supervivencia

1. Remuestrear discos para construir cada árbol $b = 1, \ldots, B$.
2. En cada nodo, sortear un subconjunto de m_{try} covariables.
3. Escoger un corte de supervivencia y crecer hasta el tamaño mínimo.
4. Estimar riesgo y supervivencia en cada nodo terminal; luego agregar.

$$\widehat{H}_b(t \mid x) = \sum_{t_j \le t} \frac{d_{bj}(x)}{n_{bj}(x)}.$$

$d_{bj}(x)$ y $n_{bj}(x)$ corresponden al nodo terminal al que llega x.

La aleatorización reduce la dependencia entre árboles; el promedio estabiliza.

Ishwaran et al. (2008). OOB: discos no usados para construir un árbol. El esquema exacto de remuestreo depende de la implementación.

RSF: curvas y score de mortalidad

Las curvas de cada árbol se agregan por separado:

$$\widehat{H}_{\mathrm{RSF}}(t \mid x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{H}_b(t \mid x), \quad \widehat{S}_{\mathrm{RSF}}(t \mid x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{S}_b(t \mid x).$$

El score de mortalidad de `randomForestSRC` es

$$r(x) = \sum_{j=1}^{J} \widehat{H}_{\mathrm{RSF}}(t_j \mid x).$$

Mayor $r(x)$ indica mayor mortalidad acumulada. No es una probabilidad y depende de la grilla temporal utilizada.

En general, $\widehat{S}_{\mathrm{RSF}} \neq \exp(-\widehat{H}_{\mathrm{RSF}})$**.**

No intercambiar promedio y exponencial. `randomForestSRC`: supervivencia y mortalidad.

Ajuste RSF e importancia por permutación

```
bosque <- randomForestSRC::rfsrc(formula_surv, data = train,
  ntree = 150, nodesize = 100, mtry = mtry_elegido,
  nsplit = 5, importance = "permute", seed = 20260704)
pred <- predict(bosque, newdata = test)
riesgo <- pred$predicted
```

La importancia de X_j compara el error OOB antes y después de romper su información:

$$\mathrm{VIMP}_j = \mathrm{Err}_{\mathrm{OOB}}(X_j \text{ permutado}) - \mathrm{Err}_{\mathrm{OOB}}(\text{datos originales}).$$

Importancia predictiva no equivale a efecto causal ni a coeficiente.

En este ajuste, 150 árboles y tamaño de nodo 100 fueron configuraciones fijas, no un óptimo buscado.

Boosting con pérdida de Cox

Se sustituye $x^\top \beta$ por una función flexible $f(x)$ y se minimiza

$$\mathcal{L}(f) = -\sum_{i:\Delta_i=1} \left[f(x_i) - \log \sum_{k \in \mathcal{R}_i} e^{f(x_k)} \right].$$

Sin empates, el gradiente negativo para la observación i es

$$u_i = -\frac{\partial \mathcal{L}}{\partial f_i} = \Delta_i - e^{f_i} \sum_{j:\Delta_j=1,\, Y_j \le Y_i} \frac{1}{\sum_{k \in \mathcal{R}_j} e^{f_k}}.$$

Cada etapa aproxima estos pseudorresiduos mediante un árbol pequeño.

La censura entra en los conjuntos de riesgo, no como una etiqueta negativa.

Fórmulas sin empates para la exposición; el algoritmo trata los tiempos empatados.

Boosting: actualización y regularización

Si $g_m(x)$ aproxima el gradiente en la etapa m, la actualización es

$$f_m(x) = f_{m-1}(x) + \nu \rho_m g_m(x), \qquad 0 < \nu \leq 1.$$

- Profundidad: limita interacciones de cada árbol.
- Tasa ν: controla el aporte de cada nueva etapa.
- Número de etapas: equilibra ajuste y sobreajuste; se selecciona con validación.

$$h(t \mid x) = h_0(t) e^{f_M(x)}.$$

Efectos no lineales no implican riesgos no proporcionales.

El ajuste utiliza boosting con árboles y pérdida Cox (`gbm`); `CoxBoost` es otro algoritmo, no el ajuste reportado.

Boosting en R: cuándo detenerse

```
boost <- gbm::gbm(formula_surv, data = train, distribution = "coxph",
  n.trees = 500, interaction.depth = 2, shrinkage = 0.02,
  n.minobsinnode = 100, bag.fraction = 0.7, train.fraction = 1)
lp_valid <- predict(boost, newdata = valid, n.trees = 400, type = "link")
lp_test  <- predict(boost, newdata = test,  n.trees = 400, type = "link")
```

$$\widehat{M} = \arg \max_{M \in \{100,200,300,400,500\}} \widehat{C}_H(\text{validación}; f_M) = 400.$$

Se mantienen profundidad 2, tasa 0.02 y fracción de remuestreo 0.7.

Más etapas no garantizan mejor capacidad predictiva fuera del ajuste.

SVM: una pérdida que respeta la censura

En la variante de regresión, $f_i = w^\top \phi(x_i) + b$ predice el tiempo observado en su escala original:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 + \gamma \sum_i \left[(Y_i - f_i)_+ + \Delta_i (f_i - Y_i)_+\right], \qquad (a)_+ = \max(a, 0).$$

- Si $\Delta_i = 1$, la pérdida es $|Y_i - f_i|$: penaliza ambos lados.
- Si $\Delta_i = 0$, solo se penaliza $f_i < Y_i$: se sabe que $T_i > Y_i$.
- γ equilibra complejidad y ajuste; no es el ancho del kernel.

Un censurado no debe tratarse como una falla en Y_i.

Forma primal equivalente a la regresión de `survivalsvm`, sin banda ε. Fouodo et al. (2018).

Kernels: introducir relaciones no lineales

El truco del kernel utiliza $K(x, z) = \langle \phi(x), \phi(z) \rangle$ sin construir explícitamente ϕ.

$$K_{\text{lineal}}(x, z) = x^\top z,$$

$$K_{\text{aditivo}}(x, z) = \sum_{j=1}^{p} k_j(x_j, z_j),$$

$$K_{\text{RBF}}(x, z) = \exp\{-\|x - z\|^2/(2\ell^2)\}.$$

El aditivo combina componentes univariadas; su definición concreta depende de la implementación y el tipo de variable.

En la aplicación se compararon solo kernel lineal y aditivo.

RBF se muestra como alternativa teórica. Existen también SVM de ranking; no son la variante ajustada aquí.

SVM en R: ajuste y costo computacional

```
svm <- survivalsvm::survivalsvm(formula_surv, data = train_svm,
  type = "regression", kernel = "add_kernel", gamma.mu = 0.1)
tiempo <- predict(svm, newdata = test)$predicted
riesgo <- -as.numeric(tiempo)
```

El benchmark utilizó **400 discos**: 100 fallas y 300 censurados, tomados solo de entrenamiento.

$$17\,442^2 = 304\,223\,364 \quad \text{entradas} \approx 3.04 \times 10^8; \qquad 8n^2 \approx 2.43 \text{ GB}.$$

Solo la matriz kernel completa ya requiere ese orden de memoria, además del solver y sus copias.

La submuestra hace viable una comparación exploratoria, no equivalente.

`gamma.mu=0.1` fijo. Las fallas están sobrerrepresentadas (25 %); la submuestra no es “balanceada”.

Las funciones centrales en R

Tarea	Función principal	Salida de interés
Kaplan–Meier	`survival::survfit()`	Curva marginal
AFT paramétrico	`survival::survreg()`	Cuantiles de tiempo
Cox clásico	`survival::coxph()`	Log-riesgo y HR
Cox LASSO/Ridge	`glmnet::glmnet()`	Trayectoria en λ
Árbol	`rpart::rpart()`	Riesgo por nodo
RSF	`randomForestSRC::rfsrc()`	Curvas y mortalidad
Boosting Cox	`gbm::gbm()`	Log-riesgo
SVM censurado	`survivalsvm::survivalsvm()`	Tiempo / score
Discriminación	`survival::concordance()`	C-index

Respuesta común: `Surv(time_days, event)`**, con evento 0/1.**

Qué se ajustó y qué se seleccionó

Modelo	Configuración	Selección realizada
AFT	Weibull y log-normal	MLE; sin penalizar
Cox PH	Efectos principales; Efron	Verosimilitud parcial
Cox LASSO / Ridge	80 valores de λ	Mayor C-index de validación
Árbol	Profundidad ≤ 5; nodo ≥ 80	Poda por error CV de `rpart`
RSF	150 árboles; nodo 100	Configuración fija
Boosting	Profundidad 2; tasa .02	400 etapas por validación
SVM	$n = 400$; $\gamma = .1$	Aditivo frente a lineal, por validación

La búsqueda de hiperparámetros no fue igual para todos.

Comparación exploratoria con una partición. AFT penalizado y kernel RBF no forman parte de los resultados.

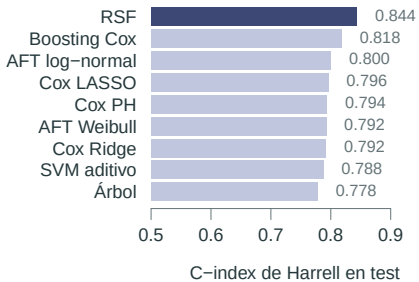
Resultados: concordancia predictiva

Modelo	Entrenamiento	Validación	Test
Random survival forest	.902	.859	**.844**
Boosting Cox	.841	.846	.818
AFT log-normal	.811	.817	.800
Cox LASSO	.806	.812	.796
Cox PH	.804	.811	.794
AFT Weibull	.803	.812	.792
Cox Ridge	.805	.809	.792
SVM aditivo ($n = 400$)	.789	.767	.788
Árbol de supervivencia	.813	.801	.778

RSF lidera en esta partición; no implica superioridad general.

Harrell; 145 fallas en test. Entrenamiento: resustitución. Sin intervalos ni particiones repetidas.

La diferencia frente al Cox de referencia



El eje se inicia en 0.5 y no en cero. Mismo test para todos; SVM entrenado con submuestra.

RSF frente a Cox PH

$$.844 - .794 \approx .050.$$

Boosting: mejora de .024 frente a Cox PH.

Diferencias pequeñas requieren cuantificar incertidumbre.

Cox: lectura de asociaciones

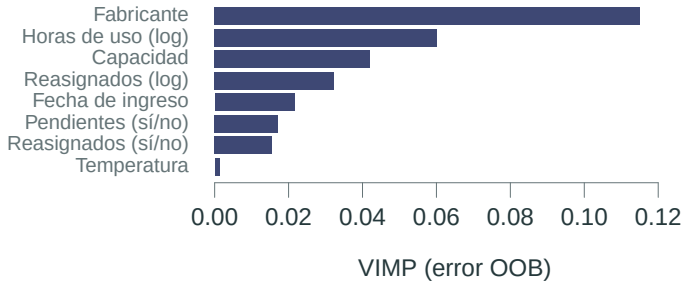
Covariable	HR estimado	Lectura condicional
Seagate frente a Hitachi	7.236	Mayor tasa observada
Sectores pendientes: sí	4.456	Señal de deterioro
$\log(1 + \text{reasignados})$	1.389	Mayor riesgo
$\log(1 + \text{horas})$	.731	Asociación inversa

Fabricante resume familias de equipos y condiciones de uso. La asociación de horas puede reflejar selección de discos que ya sobrevivieron.

No son efectos causales ni comparaciones de marcas actuales.

Modelo Cox multivariable del análisis original; referencia de fabricante: Hitachi. HR sin intervalos en este resumen.

RSF: qué variables sostienen la predicción

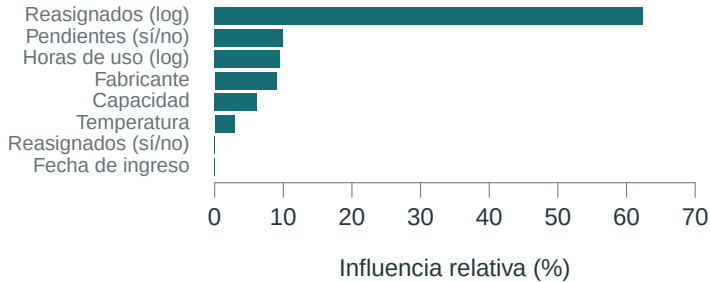


Fabricante, horas y capacidad aportan más al error predictivo OOB en esta configuración.

Permutar una variable mide dependencia predictiva, no dirección del efecto.

La correlación entre covariables puede repartir o distorsionar la importancia. No son porcentajes.

Boosting: otra medida de importancia

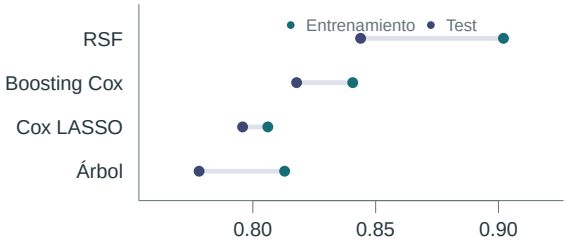


Los sectores reasignados concentran 62.4 % de la influencia relativa del ajuste.

Influencia del boosting e importancia por permutación no son la misma escala.

Influencia relativa: contribución de las variables a las mejoras de ajuste de los árboles; no es efecto causal ni fracción de fallas explicadas.

La brecha entre entrenamiento y test



C−index: entrenamiento y test

RSF
Brecha de .058: el ajuste dentro de muestra es más optimista.

Regularización
Cox LASSO presenta menor brecha, aunque un C-index de test inferior.

Escoger por desempeño dentro del entrenamiento favorece el sobreajuste.

C-index de entrenamiento por resustitución. No representa error OOB ni una estimación independiente.

SVM: comparar los dos kernels

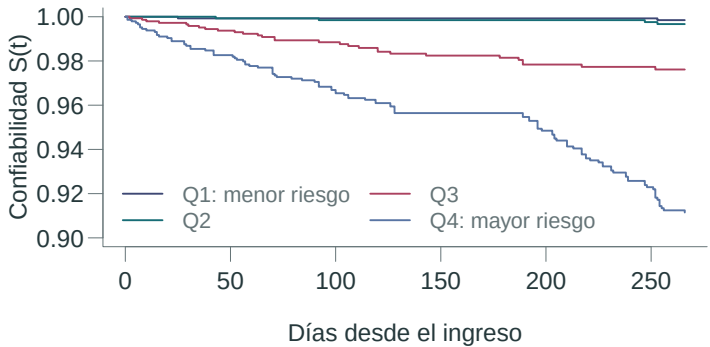
Kernel	n	Validación	Test	Ajuste (s)
Lineal	400	.564	.612	6.17
Aditivo	400	.767	.788	11.80

- Misma submuestra, $\gamma = .1$ y evaluación en los conjuntos completos.
- La selección se hizo con validación, no con el resultado de test.
- El tiempo mostrado depende del equipo y no incluye todo el flujo de datos.

La comparación con RSF mezcla algoritmo y cantidad/composición de entrenamiento.

Semilla de submuestreo 20260721. No se ajustó una búsqueda extensa de γ ni kernel RBF.

Del score a grupos de riesgo



El test se divide en cuartiles del score RSF.

Q4 concentra mayor frecuencia de falla; Q1 y Q2 mantienen alta supervivencia.

Son curvas KM de grupos, no curvas individuales del bosque.

Cuartiles construidos en test. Curvas del análisis original, redibujadas desde sus coordenadas vectoriales para traducir los rótulos.

Estratificar para priorizar inspecciones

Cuartil	Discos	Fallas	Proporción cruda	$1 - \widehat{S}_{KM}(180)$
Q1: menor riesgo	1 454	2	0.138 %	0.074 %
Q2	1 454	4	0.275 %	0.154 %
Q3	1 454	29	1.994 %	1.855 %
Q4: mayor riesgo	1 453	110	7.571 %	4.357 %

El cuartil de mayor riesgo contiene $110/145 = 75.9\,\%$ de las fallas del test en aproximadamente 25 % de los discos.

Es una señal para priorizar inspección, no una regla de reemplazo validada.

Proporción cruda: todo el seguimiento; KM: horizonte 180 días e incorpora censura. Ninguna demuestra calibración individual.

Antes de una recomendación operativa

Limitación observada	Siguiente comprobación necesaria
Imputación global	Reestimar transformaciones solo con entrenamiento
Valores pos-ingreso	Asegurar medición basal; evitar información futura
Partición única	Validar en un período posterior y repetir evaluación
Censura muy alta	Examinar soporte temporal y sensibilidad de la concordancia
Sin incertidumbre	Cuantificarla y revisar calibración por horizonte
Cuartiles del test	Fijar umbrales antes de evaluar y considerar costos

Los números presentados son un benchmark exploratorio, no validación de despliegue.

Una concordancia IPCW como Uno, restringida a un horizonte con soporte, es una alternativa futura; no fue calculada aquí.

Una secuencia para seguir trabajando

1. Definir falla, origen temporal, horizonte y covariables disponibles.
2. Describir censura y supervivencia; separar datos antes de preprocesar.
3. Ajustar un referente interpretable: AFT o Cox.
4. Añadir regularización y flexibilidad; seleccionar sin usar el test.
5. Evaluar ordenamiento, incertidumbre y probabilidades antes de decidir.

Hallazgos de esta aplicación

RSF y boosting capturaron estructura predictiva adicional; la utilidad industrial requiere comprobar estabilidad temporal y costos de actuación.

Predecir riesgo ayuda a priorizar; decidir mantenimiento exige un paso más.

Lecturas y documentación

- Cox, D. R. (1972). Regression models and life-tables. *JRSS B*, 34, 187–220.
- Tibshirani, R. (1997). The lasso method for variable selection in the Cox model. *Statistics in Medicine*, 16, 385–395.
- Ishwaran, H. et al. (2008). Random survival forests. *Annals of Applied Statistics*, 2, 841–860.
- Bühlmann, P. y Hothorn, T. (2007). Boosting algorithms: regularization, prediction and model fitting. *Statistical Science*, 22, 477–505.
- Fouodo, C. J. et al. (2018). Support vector machines for survival analysis with R. *The R Journal*, 10(1).
- Harrell, F. E. et al. (1996). Multivariable prognostic models. *Statistics in Medicine*, 15, 361–387.

Datos y funciones

Backblaze Drive Stats RSF `rpart.exp` Cox penalizado en `glmnet`